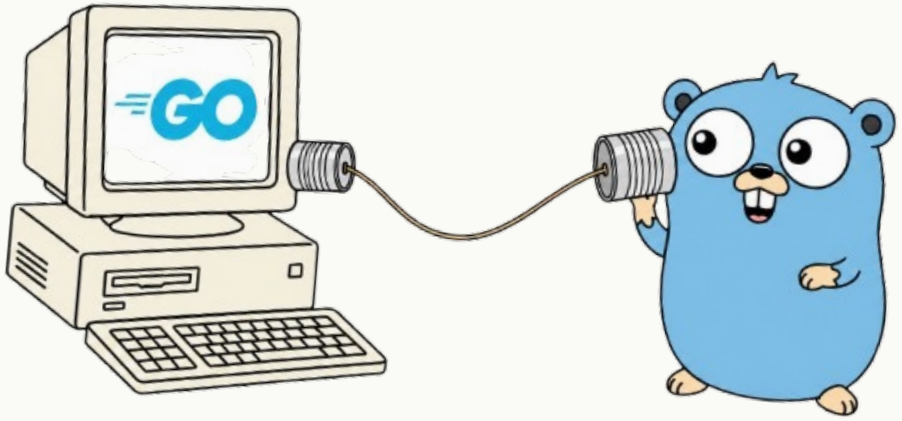
GO AMSTERDAM MEETUP · LIGHTNING TALK

Real-time voice AI agents in Go

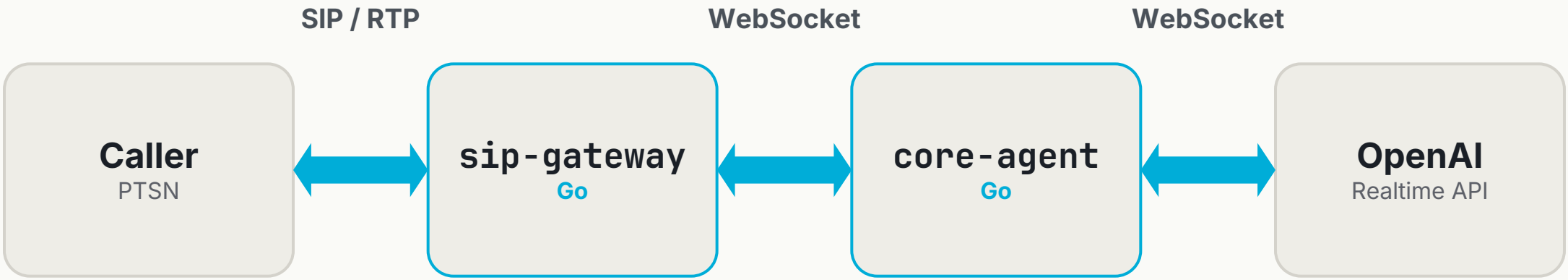
David Stotijn · Stellar · April 29, 2026



"Hi, this is Gophy,
your virtual assistant.
How can I help today?"

Running “voice-to-voice” AI agents

STELLAR AGENT PLATFORM (SIMPLIFIED)



Audio both ways, the whole call.

Hangup can come from any arrow.

WHY GO FITS

A call is `goroutines + channels + a `Context`.`

- One **`goroutine`** per audio leg — cheap, no thread pool to tune.
- **`Channels`** carry the streams — backpressure built in.
- **`` `context.Context` `** propagates hangup — SIP BYE, WS close, timeout: one cancel unwinds the call.
- Stdlib is enough: *`net/http`*, *`encoding/json`*, *`sync`*.

SIP + MEDIA: PURE GO

Look ma, no C (well almost no C)

`emiago/sipgo`
SIP signaling

`emiago/diago`
Dialogs, calls, media bridging

Underneath: Pion (RTP/RTCP/SRTP/DTLS), zaf/g711, hraban/opus.
Opus is the only CGO module. Everything else is Go.

OOPS… 1 / 2

One channel to rule them all

Buffered *chan(128)*. Audio (~50/s) and control events share it.

```
func (s *Session) Write(ev Event) error {
    select {
    case s.out <- ev:
        return nil
    default:        // non-blocking
        return ErrChannelFull
    }
}
```

WS stalls 2s. Buffer fills with audio. Function results & guardrails: **dropped silently.**

OOPS… 2 / 2

The fix that wasn't two channels

Splitting into two channels breaks FIFO between *append* and *commit*.

Audio
Non-blocking
Drop on full - latency wins.

Control
Wait 500ms
Then drop - stale anyway.

Tool call output
Retry 3x
Must reach the model.

Same channel. Sender discriminates by message class.
Don't mix data & control plane unless ordering forces you to — then handle at the sender.

Thanks! Questions?

David Stotijn · Stellar

github.com/dstotijn
linkedin.com/in/dstotijn

www.stellarcs.ai

We're hiring engineers 😬